TroveTrends Frontend Architecture

TroveTrends Frontend Architecture

Table of Contents

1. [Overview](#overview)
2. [Technology Stack](#technology-stack)
3. [System Architecture](#system-architecture)
4. [Directory Structure](#directory-structure)
5. [Data Flow](#data-flow)
6. [State Management](#state-management)
7. [Rendering Strategy](#rendering-strategy)
8. [External Integrations](#external-integrations)

Overview

TroveTrends is a Next.js 13+ application built with the App Router architecture. The platform serves as a multi-category content publishing system with integrated email marketing, lead generation, and analytics capabilities.

Core Principles

- **Server-First Rendering**: Leverages Next.js App Router for optimal SEO and performance
- **Type Safety**: Full TypeScript implementation with strict mode enabled
- **Component Modularity**: Feature-based component organization
- **Security by Design**: Input validation at multiple layers (client, API, database)

Technology Stack

Frontend Framework

Technology	Version	Purpose
Next.js	13.5.1	React framework with App Router
React	18.2.0	UI component library
TypeScript	5.2.2	Type-safe JavaScript

UI and Styling

Technology	Version	Purpose
Tailwind CSS	3.3.3	Utility-first CSS framework
shadcn/ui	-	Radix UI-based component library
Lucide React	0.446.0	Icon library
Tailwind Merge	2.5.2	Class name conflict resolution
CVA	-	Component variant management

Backend Services

Technology	Version	Purpose
Firebase	11.3.1	Client SDK for Firestore
Firebase Admin	13.2.0	Server-side Firebase operations
SendGrid	8.1.5	Transactional email delivery

Form Handling

Technology	Version	Purpose
React Hook Form	7.53.0	Form state management
Zod	3.23.8	Schema validation
@hookform/resolvers	3.9.0	Zod integration for RHF

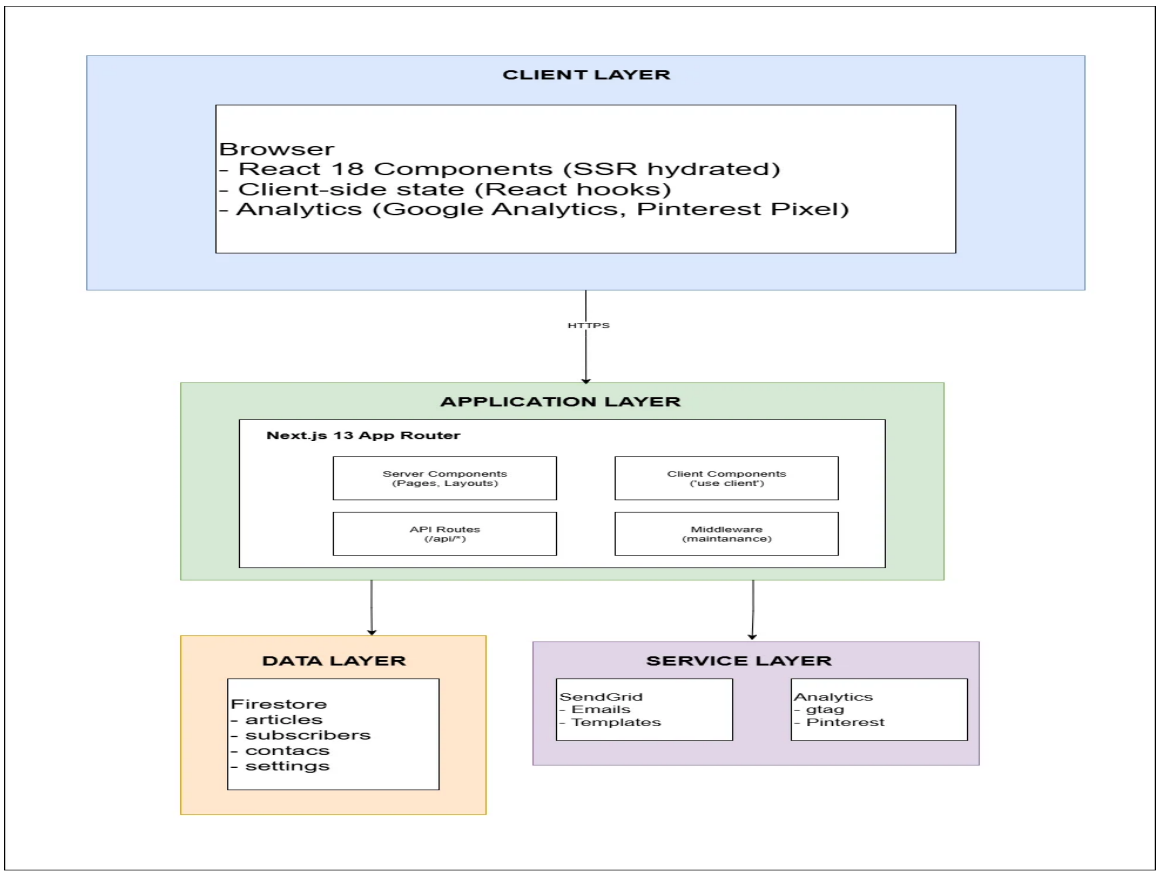
Data Visualization

Technology	Version	Purpose
Recharts	2.12.7	Chart components
Embla Carousel	8.3.0	Carousel functionality

Development Tools

Technology	Version	Purpose
ESLint	8.49.0	Code linting
Jest	30.0.4	Testing framework
ts-jest	29.4.0	TypeScript Jest support

System Architecture



Layer Responsibilities

Client Layer

- Renders React components after SSR hydration
- Manages client-side interactivity
- Tracks user engagement and analytics events

Application Layer

- Handles HTTP requests via Next.js server

- Renders pages with SSR for SEO optimization
- Processes API requests through route handlers
- Applies middleware for cross-cutting concerns

Data Layer

- Persists application data in Firestore
- Provides real-time data synchronization
- Enforces security rules at database level

Service Layer

- Delivers transactional emails via SendGrid
- Tracks analytics events to external providers

Directory Structure

```
trovetrends-frontend/
|
+-- app/                          # Next.js App Router
|   +-- (pages)/                  # Route segments
|   |   +-- blog/[slug]/          # Dynamic article pages
|   |   +-- categories/           # Category browsing
|   |   +-- calculator/           # Finance calculator
|   |   +-- quiz/                 # Interactive quiz
|   |   +-- contact/              # Contact page
|   |   +-- trending/             # Trending articles
|   |   +-- featured/             # Featured articles
|   |   +-- reviews/              # Product reviews
|   |   +-- about/                # About page
|   |   +-- privacy-policy/       # Legal pages
|   |   +-- terms-of-service/
|   |   +-- full-disclosure/
|   |   +-- rss-feeds/            # RSS feed listing
|   |   +-- maintenance/          # Maintenance page
|   |   +-- subscription-*/       # Subscription flows
|   |   +-- unsubscribe/
|   |   +-- thank-you/
|   |
|   +-- api/                      # API Route Handlers
|   |   +-- newsletter/           # Newsletter subscription
|   |   +-- confirm-subscription/ # Email confirmation
|   |   +-- unsubscribe/          # Unsubscribe handler
|   |   +-- contact/              # Contact form
|   |   +-- featured-article/     # Featured article API
|   |   +-- download-pdf/         # Secure PDF downloads
|   |   +-- generate-pdf-token/   # JWT token generation
|   |   +-- sitemap.xml/          # Dynamic sitemap
|   |   +-- rss.xml/              # Dynamic RSS feed
|   |   +-- system-status/        # Health check
|   |   +-- system-settings/      # App configuration
|   |
|   +-- globals.css               # Global styles
|   +-- layout.tsx                # Root layout
|   +-- page.tsx                  # Home page
|
+-- components/                   # React Components
|   +-- Articles/                 # Article templates and content
|   +-- Calculator/               # Finance calculator components
|   +-- Header/                   # Navigation header
|   +-- Footer/                   # Site footer
|   +-- HeroSection/              # Hero banners
|   +-- Newsletter/               # Newsletter signup
```

```
|   +-- PopupLeadMagnet/          # Lead capture popups
|   +-- FeatureCategories/        # Category display
|   +-- TrendingArticles/         # Trending content
|   +-- RelatedArticles/          # Related content
|   +-- Quiz/                     # Quiz interface
|   +-- ContactForm/              # Contact form
|   +-- ReadingTracker/           # Engagement tracking
|   +-- SocialMedia/              # Social sharing
|   +-- Sidebar/                  # Article sidebars
|   +-- MainContent/              # Article body
|   +-- Layouts/                  # Layout wrappers
|   +-- Maintenance/              # Maintenance UI
|   +-- BackToTopButton/          # Scroll utility
|   +-- ui/                       # shadcn/ui components
|
+-- utils/                        # Utilities
|   +-- firebase/                 # Firebase configuration
|   +-- services/                 # Business logic services
|   +-- hooks/                    # Custom React hooks
|   +-- helpers/                  # Helper functions
|
+-- lib/                          # Shared libraries
|   +-- utils.ts                  # Common utilities
|
+-- types/                        # TypeScript definitions
|   +-- index.ts                  # Type exports
|
+-- data/                         # Static data
|   +-- blogData.ts               # Article content
|   +-- categories.ts             # Category definitions
|
+-- scripts/                      # Maintenance scripts
|   +-- migrate*.ts               # Data migration
|   +-- testQueries.ts            # Query testing
|
+-- public/                       # Static assets
|   +-- images/                   # Image files
|   +-- fonts/                    # Font files
|   +-- manifest.json             # PWA manifest
|
+-- Configuration Files
    +-- package.json              # Dependencies
    +-- tsconfig.json             # TypeScript config
    +-- tailwind.config.ts        # Tailwind config
    +-- next.config.mjs           # Next.js config
    +-- postcss.config.js         # PostCSS config
    +-- components.json           # shadcn/ui config
    +-- middleware.ts             # Request middleware
    +-- firebase.json             # Firebase CLI config
    +-- firestore.rules           # Security rules
```

Data Flow

Article Page Request Flow

```
1. Browser Request
   GET /blog/article-slug
         |
         v
2. Middleware Check
   - Maintenance mode?
   - Redirect if enabled
         |
```

```
        v
3. Server Component Render
   - Load article from blogData.ts
   - Fetch related articles
   - Generate metadata
        |
        v
4. HTML Response
   - SSR rendered page
   - Hydration scripts
        |
        v
5. Client Hydration
   - React takes over
   - Event listeners attached
   - Analytics initialized
        |
        v
6. User Interaction
   - Reading tracker active
   - Newsletter popup (45s delay)
   - Scroll tracking
```

Newsletter Subscription Flow

```
1. User Submits Email
   POST /api/newsletter
   { email, source }
        |
        v
2. Server Validation
   - Email format check
   - Duplicate check in Firestore
        |
        v
3. Database Write
   - Create subscriber record
   - Status: pending_confirmation
   - Generate confirmation token
        |
        v
4. Email Dispatch
   - SendGrid API call
   - Confirmation link with token
        |
        v
5. User Confirms
   GET /api/confirm-subscription?token=xxx
        |
        v
6. Status Update
   - Update Firestore record
   - Status: confirmed
   - Send welcome email
```

State Management

Approach

The application uses React's built-in state management without external libraries like Redux or Zustand.

State Patterns

Local Component State

```
// Simple UI state
const [isOpen, setIsOpen] = useState(false);
```

Custom Hooks for Complex State

```
// Finance calculator state
const { inputs, setInputs, results } = useCalculator();
```

Server State

- Firestore real-time listeners for dynamic data
- Static imports for blog content

State Locations

State Type	Location	Example
UI State	Component useState	Modal open/close
Form State	React Hook Form	Newsletter signup
Calculator State	useCalculator hook	Financial inputs
Article Data	Static imports	blogData.ts
Subscriber Data	Firestore	Real-time sync

Rendering Strategy

Server Components (Default)

Pages and layouts are server components by default:

```
// app/blog/[slug]/page.tsx
export default function ArticlePage({ params }) {
  // Runs on server
  const article = getArticle(params.slug);
  return <ArticleLayout article={article} />;
}
```

Client Components

Interactive components use the 'use client' directive:

```
// components/Newsletter/NewsletterForm.tsx
'use client';

export function NewsletterForm() {
  const [email, setEmail] = useState('');
  // Client-side interactivity
}
```

Rendering Modes

Page Type	Rendering	Reason
Article pages	SSR	SEO optimization
Category pages	SSR	SEO optimization
Calculator	CSR	Heavy interactivity
Quiz	CSR	Interactive state
API routes	Server	Data operations

External Integrations

Firebase Firestore

Collections

Collection	Purpose	Access
newsletter_subscribers	Email subscriptions	Server-side
contacts	Contact form submissions	Server-side
articles	Article content (future)	Server-side
systemSettings	App configuration	Server-side

Connection

```
// Client SDK (read operations)
import { db } from '@/utils/firebase/config';

// Admin SDK (write operations)
import { adminDb } from '@/utils/firebase/admin';
```

SendGrid

Email Types

Template	Trigger	Purpose
Confirmation	Subscription	Verify email ownership
Welcome	Confirmation	Onboard new subscriber
Unsubscribe	Request	Confirm removal

Analytics

Google Analytics

- Page views tracked automatically
- Custom events for conversions
- Affiliate link tracking

Pinterest Pixel

- Tag ID: 2613373434888
- Conversion tracking
- Audience building

Performance Considerations

Build Optimization

- SWC minification enabled
- Standalone output for containerization
- Image optimization via next/image

Runtime Optimization

- Server components reduce client bundle
- Dynamic imports for heavy components
- Static generation where possible

Caching Strategy

- Static assets cached indefinitely
- API responses use appropriate cache headers
- Firestore provides real-time sync with local caching

Related Documentation

- [API Reference](./API.md)
- [Security Model](./SECURITY.md)
- [Development Guide](./DEVELOPMENT.md)
- [Component Documentation](./COMPONENTS.md)
- [Deployment Guide](./DEPLOYMENT.md)